

GPU-Accelerated Image Processing Library Assessment

As part of our evaluation process for the *AI Software Engineer* position, we invite you to create a high-performance image processing library.

This project involves developing a memory-efficient image processing library implemented in C++, with GPU acceleration capabilities using CUDA. You will implement fundamental image processing operations and create a clean, well-documented API. To showcase cross-language integration skills, you'll develop Python bindings that provide a Pythonic interface to the library.

The assessment emphasizes software engineering best practices. You'll set up a CMake-based build system, write comprehensive unit tests, and provide clear documentation. Performance is a key consideration – your submission should include benchmarks comparing CPU and GPU implementations.

Your final submission should include the source code, build configuration, documentation, and test suite. This will allow us to evaluate your proficiency in C++ memory management, CUDA programming (optional but recommended), Python integration, build system configuration, and overall code organization.

We look forward to reviewing your implementation and discussing your design choices in our follow-up conversation.

Requirements:

- Implement the following basic image processing operations: grayscale conversion, blur, edge detection, and histogram equalization.
- Use modern C++ features (C++17 or C++20) and demonstrate proper memory management
- Implement a custom image class that efficiently handles large images
- Create CUDA implementations for at least 1 of the above algorithms (optional but recommended)
- Implement a mechanism to choose between CPU and GPU implementations based on available hardware
- Utilize [nanobind](#) to create Python binding for at least 1 of the image processing operations in the C++ library
- Provide an example script that exercises library functions through Python
- Build system to be managed by CMake (you may use Conan or vcpkg in addition)

Deliverables:

- C++ library and Python API with above requirements satisfied
- Clear documentation for the library
- Unit tests for core areas (project does not need to be fully unit tested) using PyTest (Python) and Google Test (C++)

- Screen recorded demo/walkthrough of the library